

ระบบจองห้องพักโรงแรมแบบ Client-Server

การออกแบบโปรโตคอลและสถาปัตยกรรมระบบ

1. วัตถุประสงค์

โปรแกรมนี้จำลองระบบจองห้องพักโรงแรมที่ต้องรองรับผู้ใช้หลายคนพยายามจองห้องเดียวกันในเวลาไล่เลี่ยกัน โดยระบบต้องรับประกันว่ามีผู้ใช้เพียงคนเดียวเท่านั้นที่สามารถจองห้องนั้นได้สำเร็จ เพื่อป้องกันปัญหา race condition และการจองซ้ำ (double booking) ซึ่งเป็นปัญหาที่พบได้จริงในระบบ ที่มีผู้ใช้งานพร้อมกันจำนวนมาก

2. คุณลักษณะของระบบ

หัวข้อ	รายละเอียด
สถาปัตยกรรม	Client-Server แบบ centralized
จำนวน client	รองรับหลาย client เชื่อมต่อพร้อมกัน (concurrent)
รูปแบบ concurrency	1 thread ต่อ 1 client connection บนฝั่ง server
การเก็บข้อมูล	In-memory (dictionary) ไม่ใช้ฐานข้อมูลจริง
กลไกป้องกันการชนกัน	threading.Lock() แยกต่อห้อง (per-room lock)
กลไกพิเศษ	ล็อกห้องชั่วคราวพร้อม timeout อัตโนมัติ หากไม่ยืนยันภายในเวลาที่กำหนด

3. เหตุผลที่เลือกใช้ TCP

เลือกใช้ TCP เป็น transport layer แทน UDP เนื่องจากลักษณะของข้อมูลในระบบนี้เป็นข้อมูลธุรกรรม (transaction) ที่ต้องการความถูกต้องแม่นยำสูง มีเหตุผลสนับสนุนดังนี้

- Reliable delivery: คำสั่งอย่าง LOCK_ROOM หรือ CONFIRM_BOOKING ต้องส่งถึงปลายทางครบถ้วน หากข้อมูลสูญหายระหว่างทางจะทำให้สถานะห้องระหว่าง client และ server ไม่ตรงกัน
- Ordered delivery: ลำดับของคำสั่งมีผลโดยตรงต่อผลลัพธ์ เช่น การจองและการยกเลิกต้องเรียงลำดับให้ถูกต้อง TCP รับประกันว่าข้อมูลมาถึงตามลำดับที่ส่ง
- Connection-oriented: เหมาะกับ transaction ที่มีหลายขั้นตอนต่อเนื่องกัน (ค้นหา ล็อก ยืนยัน) เพราะการเชื่อมต่อคงอยู่ตลอดจนกว่า client จะปิดการเชื่อมต่อ
- เทียบกับ UDP ซึ่งไม่รับประกันการส่งถึงและลำดับของข้อมูล เหมาะกับงานที่เน้นความเร็วและทนต่อการสูญหายของข้อมูลได้ เช่น video streaming จึงไม่เหมาะกับงานด้านธุรกรรมแบบนี้

4. การออกแบบโปรโตคอล (HBP)

รูปแบบข้อความสื่อสารระหว่าง client และ server ใช้ JSON ส่งผ่าน TCP socket โดยแต่ละฝั่งจะ encode/decode ข้อความเป็น UTF-8 request จะระบุ action ที่ต้องการ และ response จะตอบกลับด้วย status code พร้อมข้อความอธิบาย

4.1 สถานะของห้องพัก (Room State)

ห้องแต่ละห้องมี 3 สถานะ ได้แก่ AVAILABLE, LOCKED และ BOOKED โดยมีการเปลี่ยนสถานะดังนี้

```
AVAILABLE --LOCK_ROOM (success)--> LOCKED --CONFIRM_BOOKING--> BOOKED
LOCKED --CANCEL_LOCK--> AVAILABLE
LOCKED --timeout (30s)--> AVAILABLE
```

การเปลี่ยนสถานะของแต่ละห้องอยู่ภายใต้ critical section ที่ป้องกันด้วย threading.Lock() แยกต่อห้อง (ไม่ใช่ lock กลางร่วมกันทุกห้อง) เพื่อให้ client ที่จองห้องต่างกันไม่ต้องรอคิวกัน

4.2 คำสั่ง (Action) ที่รองรับ

Action	Field ที่ส่งมา	คำอธิบาย
SEARCH_ROOMS	-	ขอรายการห้องว่างทั้งหมด
LOCK_ROOM	room_id	ขอล็อกห้องชั่วคราว (30 วินาที)
CONFIRM_BOOKING	room_id	ยืนยันการจองห้องที่ล็อกไว้
CANCEL_LOCK	room_id	ยกเลิกการล็อก คืนห้องเป็นว่าง

4.3 รูปแบบข้อความ (Message Format)

ตัวอย่าง request:

```
{"action": "LOCK_ROOM", "room_id": "R101"}
```

ตัวอย่าง response กรณีล็อกสำเร็จ:

```
{"status": 202, "message": "Lock Acquired",
  "room_id": "R101", "expires_in": 30}
```

ตัวอย่าง response กรณีห้องถูกล็อกอยู่แล้ว:

```
{"status": 409, "message": "Room already locked"}
```

4.4 Status Code

Code	Message	ความหมาย
200	OK	สำเร็จทั่วไป (เช่น ค้นหาห้อง, ยกเลิกล็อก)
201	Booking Confirmed	ยืนยันการจองสำเร็จสมบูรณ์
202	Lock Acquired	ล็อกห้องสำเร็จ
409	Room already locked	ห้องถูกผู้ใช้อื่นล็อกอยู่ในขณะนั้น
410	Lock expired or room not locked	ล็อกหมดเวลาไปแล้วก่อนยืนยันการจอง
404	Room not found	ไม่พบห้องที่ระบุในระบบ
400	Bad Request / Invalid JSON	รูปแบบคำสั่งหรือข้อความไม่ถูกต้อง

4.5 กลไก Lock Timeout

เมื่อห้องถูกล็อกสำเร็จ ระบบจะเริ่มนับเวลาด้วย threading.Timer เป็นเวลา 30 วินาที หากไม่มีการยืนยัน การจอง (CONFIRM_BOOKING) ภายในเวลาดังกล่าว ระบบจะปลดล็อกห้องคืนสถานะ AVAILABLE โดยอัตโนมัติ โดยขั้นตอนนี้ยังคงต้องอยู่ภายใต้ lock ของห้องนั้น เพื่อป้องกันกรณีที่ client ส่งคำสั่งยืนยันมาพอดีในช่วง เวลาที่ timer กำลังทำงาน (race condition ระหว่าง timeout กับ confirm)

4.6 ตัวอย่าง Transaction Flow

Client	Server
--- SEARCH_ROOMS ----->	
<----- 200 OK (room list) -----	
--- LOCK_ROOM (R101) ----->	
<----- 202 Lock Acquired -----	
--- CONFIRM_BOOKING (R101) ----->	
<----- 201 Booking Confirmed -----	

หาก client อีกตัวหนึ่งพยายามส่ง LOCK_ROOM มายังห้องเดียวกันในระหว่างที่ยังถูกล็อกอยู่ server จะตอบกลับด้วย status 409 กันที โดยไม่ต้องรอ